

Publishing your site

DocsForge builds a completely static site — no database, no server-side processing, no runtime dependencies. This means you can host it anywhere that serves static files.

GitHub Pages (recommended)

The fastest way to get your docs online. DocsForge includes a ready-to-use GitHub Actions workflow.

1. Push to GitHub

Create a repository and push your docs project:

```
git init
git add .
git commit -m "Initial documentation"
git branch -M main
git remote add origin https://github.com/username/my-docs.git
git push -u origin main
```

2. Enable GitHub Pages

1. Go to your repository on GitHub
2. Click **Settings** → **Pages**
3. Under **Build and deployment**, select **Source: GitHub Actions**

3. The workflow is already included

Your DocsForge starter includes `.github/workflows/pages.yml`. It runs automatically on every push to `main`:

```
.github/workflows/pages.yml

name: Deploy Docs to GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:
```

```

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.x'

      - name: Build site
        run: docsforge build

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: site/

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4

```

That's it. Your site will be live at `https://username.github.io/my-docs/` within a minute.

Custom domain

Add a `CNAME` file to your `docs/` directory:

docs/CNAME

`docs.example.com`

Then configure your DNS provider with a CNAME record pointing to `username.github.io`.

Netlify

Drag and drop

1. Run `docsforge build`
2. Drag the `site/` folder to [Netlify Drop](#)

Git integration

1. Push your repo to GitHub
2. In Netlify, click **Add new site** → **Import an existing project**
3. Choose your GitHub repository
4. Set build command: `docsforge build`
5. Set publish directory: `site/`

Vercel

1. Push your repo to GitHub
2. In Vercel, click **Add New...** → **Project**
3. Import your repository
4. Override the build settings:
 - **Build Command:** `docsforge build`
 - **Output Directory:** `site/`

Cloudflare Pages

1. Push your repo to GitHub
2. In Cloudflare Pages, click **Create a project**
3. Connect your GitHub account and select your repository
4. Set build command: `docsforge build`
5. Set build output directory: `site/`

Render

1. Push your repo to GitHub
2. Log in to [Render Dashboard](#)

3. Click **New +** → **Static Site**
4. Connect your repository
5. Set build command: `pip install docsforge && docsforge build`
6. Set publish directory: `site/`

DigitalOcean App Platform

1. Push your repo to GitHub
2. Log in to [DigitalOcean](#)
3. Click **Apps** → **Create App**
4. Connect your repository
5. Select **Static Site** plan
6. Set build command: `pip install docsforge && docsforge build`
7. Set output directory: `site/`

Amazon S3 + CloudFront

For enterprise deployments:

Build and sync to S3

```
docsforge build
aws s3 sync site/ s3://my-docs-bucket --delete
aws cloudfront create-invalidation --distribution-id ABCD --paths "/*"
```

Docker

If you prefer containerized builds:

Dockerfile

```
FROM python:3.11-slim
WORKDIR /docs
COPY . .
RUN docsforge build
FROM nginx:alpine
COPY --from=0 /docs/site /usr/share/nginx/html
```

Offline distribution

Because DocsForge sites are fully static, you can also distribute them as a ZIP file:

```
docsforge build
zip -r my-docs.zip site/
```

Recipients can open `site/index.html` directly in their browser — no server required.

Build configuration for different hosts

Some hosts require specific settings. Adjust `docsforge.yml` accordingly:

GitHub Pages (project site)

```
site_url: https://username.github.io/repository-name/
```

GitHub Pages (user/organization site)

```
site_url: https://username.github.io/
```

Subdirectory deployment

```
site_url: https://example.com/docs/
```

The `site_url` is important for:

- Generating correct absolute URLs
- Enabling the `site_url` metadata for plugins
- Ensuring search and navigation work correctly

Next steps

- [Setup Guides](#) — Customize your site before publishing
- [Building an optimized site](#) — Enable minification and compression

 Jun 19, 2026  Jun 19, 2026